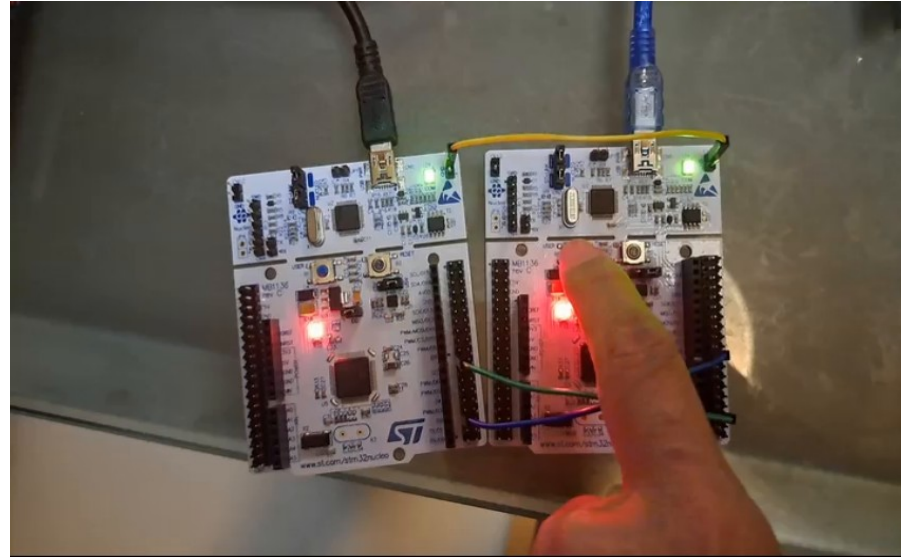


Lab 5 Brief

Communication

- UART
- SPI
- I2C

Expected Behavior



UART

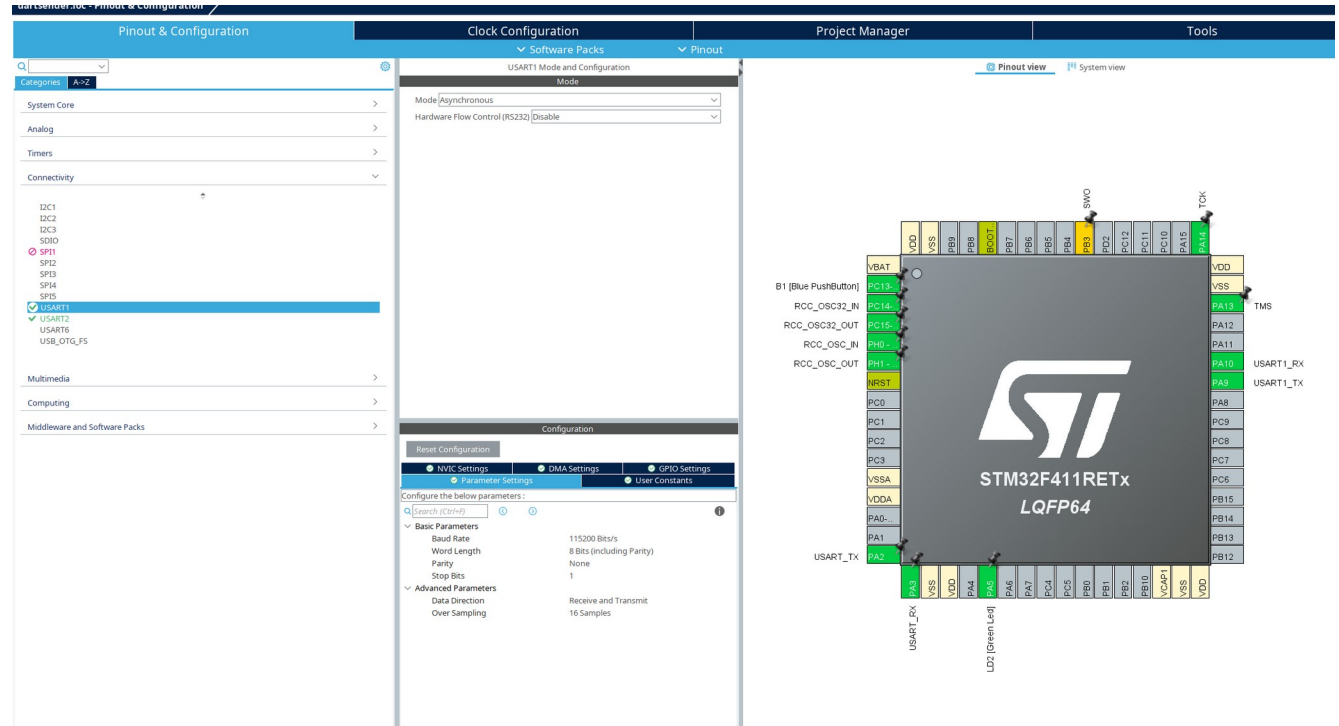
- Universal **Asynchronous** Receiver/Transmitter
- Hardware that translates between parallel and **serial** forms
- The **RXD** signal of a UART is the signal receiving the data. This will be an input and is usually connected to the TXD line of the downstream device.
- The **TXD** signal of a UART is the signal transmitting the data. This will be an output and is usually connected to the RXD line of the downstream device.

UART Setup on STM32

- STM32 have USART (Universal Synchronous Asynchronous Receiver Transmitter), which is a UART that can optionally support clock. We will be using a Asynchronous mode, so the synchronous mode is irrelevant
- There is one UART (USART2) already setup by default. This UART is connected to UART to USB. You cannot use it in the project.
- Use USART1 instead

UART Setup on STM32

- Connectivity → USART1 → Change Mode to “Asynchronous” on both boards



STM32 UART: Sending and Receiving

- Send data using HAL_UART_Transmit, i.e.

```
HAL_UART_Transmit(&huart1, "0", 1, 1000);
```

- Receiving data using HAL_UART_Receive, i.e.

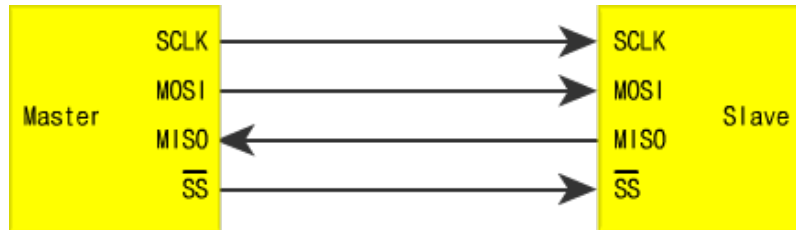
```
if (HAL_UART_Receive(&huart1, rcv, 1, 1000) == HAL_OK){
```

UART: Connection

- Connect
 - Board1-PA9 to Board2-PA10
 - Board1-PA10 to Board2-PA9 (Optional)
 - Board1-GND to Board2-GND (If you are using two different computers, i.e. one computer is connected to Board1 and another computer connected to Board2)

Serial Peripheral Interface (SPI)

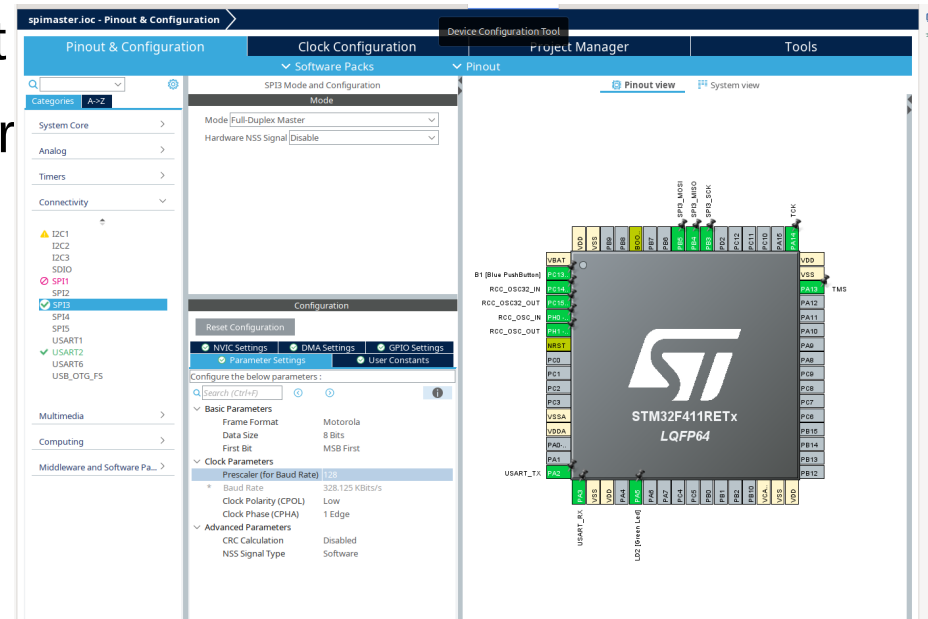
- Master – Slave
 - Only master can initiate the transfer
 - Your board 1 will have to be master
 - Your board 2 will have to be slave



Single Master and Single Slave

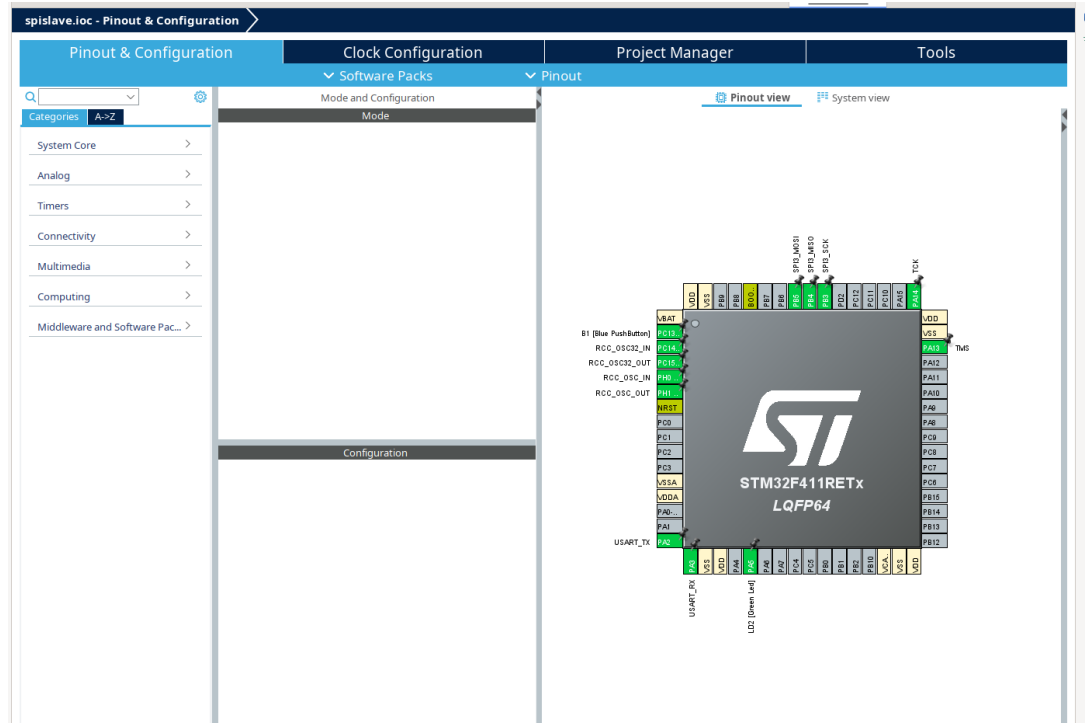
SPI on STM32

- Connectivity → SPI3 Board 1 (Other SPI will requires you to have a female to female wire)
 - Set mode to be Full-Duplex Master or Transmit only master
 - Default sending speed is a bit
 - Set Prescaler to 128 or higher



SPI on STM32

- Connectivity → SPI3 Board 2
- Set mode to be Full-Duplex Slave or Receive only slave



SPI on STM32: Connectivity

- Board 1 PB3 → Board 2 PB3
- Board 1 PB4 → Board 2 PB4 (Optional)
- Board 1 PB5 → Board 2 PB5
- (We do not need to use CS (NSS) pins.)
- Board 1 GND → Board 2 GND

SPI on STM32 (API)

- Master

```
HAL_SPI_Transmit(&hspi3, "0", 1, 1000);
```

- Slave

```
char recv[1];
```

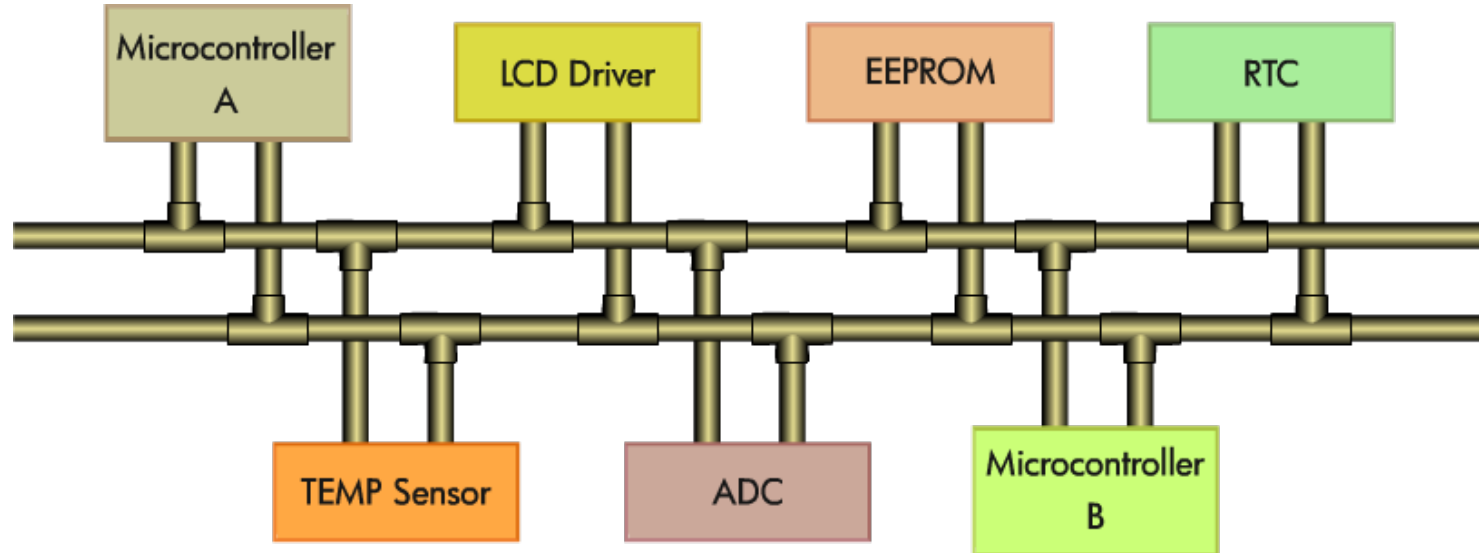
```
HAL_SPI_Receive(&hspi3, recv, 1, 1000)
```

I2C

- Synchronous Half Duplex
- Multi Master / Slave
- Two Line Interface
- Software Addressable

I2C

- Two-wired Interface
 - SDA
 - SCL

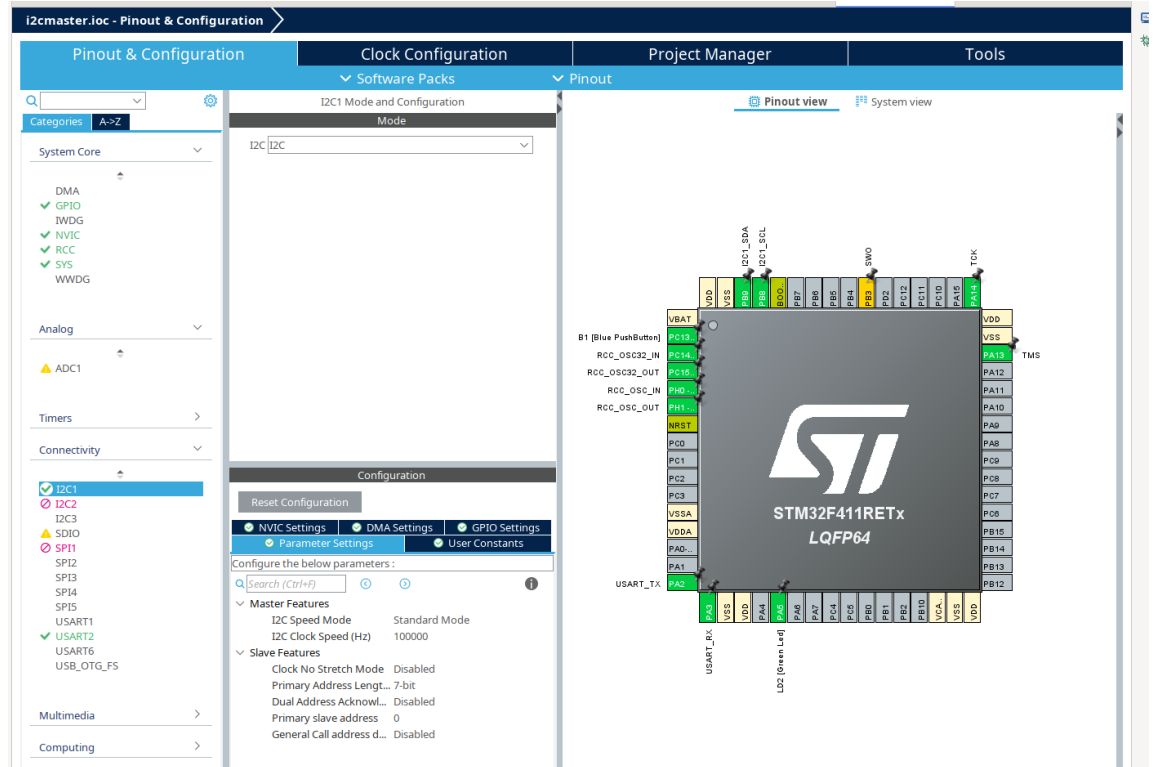


I2C Setup

- Board 1 needs to setup I2C with/without address
- Board 2 needs to setup I2C with an address
 - Slave has to have an address, set it in
Primary slave address options

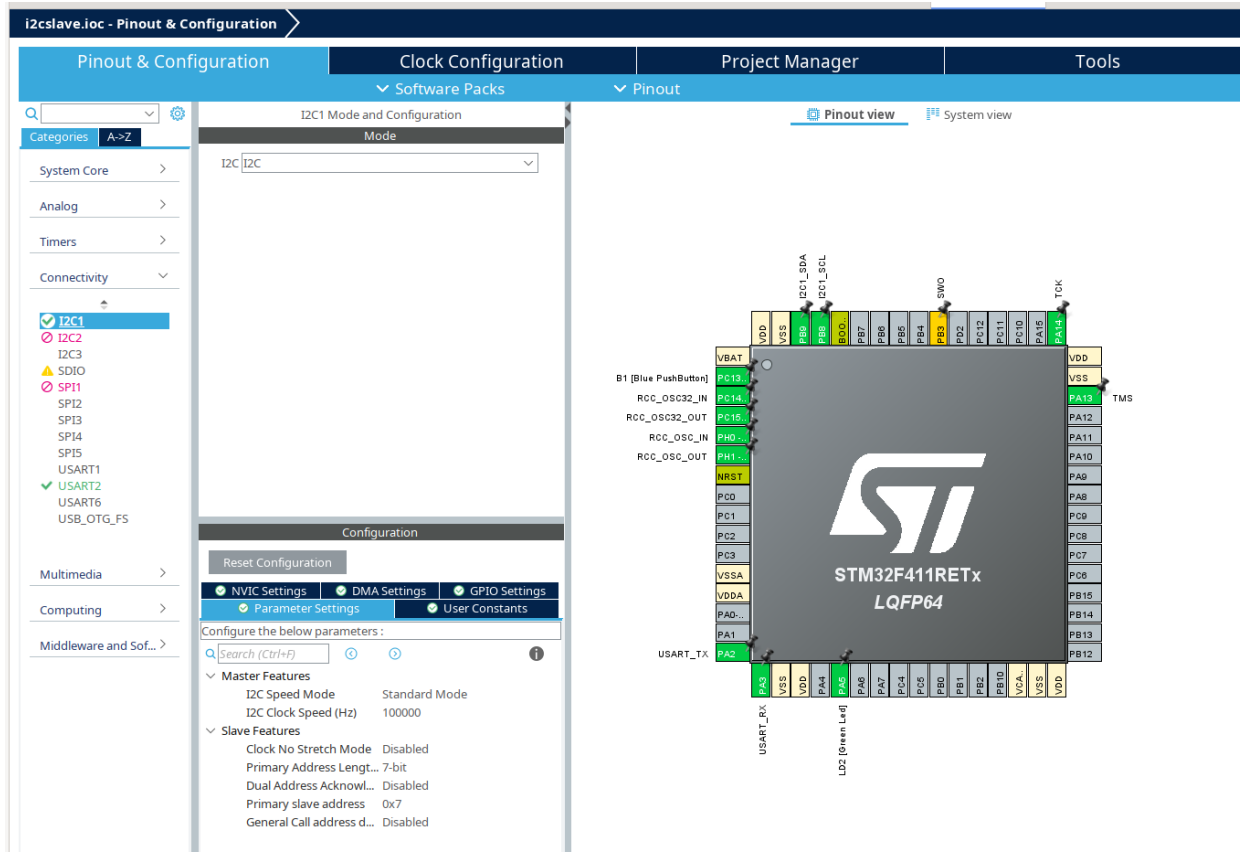
I2C Master Setup

- PB8 for SCL
- PB9 for SDA



I2C Slave Setup

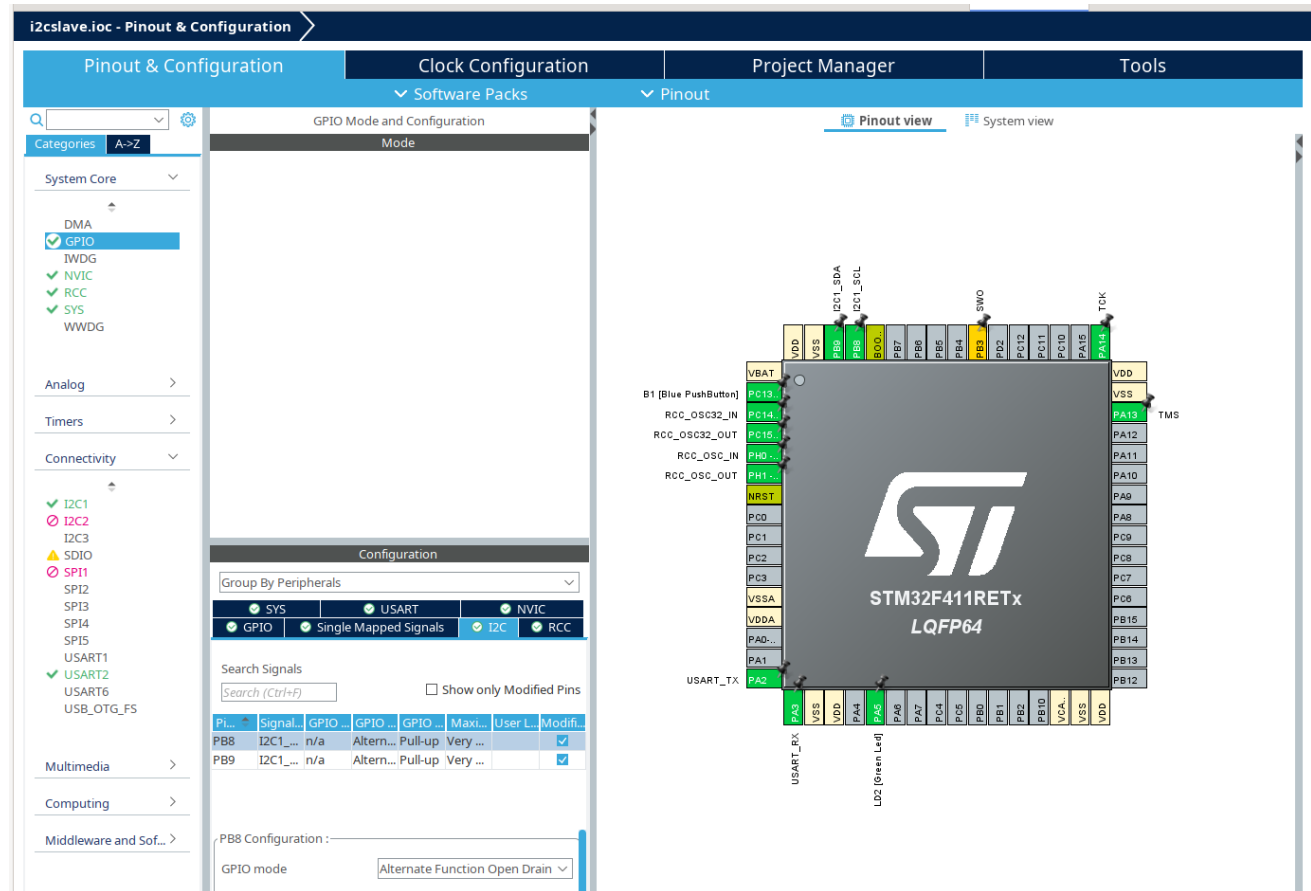
- Similar setting but I'm setting a Slave address to 0x7 (you may use a different value)



I2C More setting

- Typically, I2C signal require a pull up resistor. This is a bit complicated to setup. So, we will be using an internal pull up from STM32 Pin instead, you can set it by navigate here
 - System Core → GPIO → I2C → Select a pin → Set GPIO Pull-Up/Pull Down to Pull up
 - (Do this on both pins)

I2C GPIO Setting



I2C GPIO Setting

- Set the pins on one board is good enough, but if you cannot get to work, you can try setting it on both board

I2C Connection

- Board 1 PB8 → Board 2 PB8
- Board 1 PB9 → Board 2 PB9
- Board 1 GND → Board 2 GND

I2C API

- Receiving, I.e.

```
if (HAL_I2C_Slave_Receive(&hi2c1, recv, 1, 1000) == HAL_OK){
```

- Transmitting, a bit tricky, need address, which is a bit confusing

```
HAL_I2C_Master_Transmit(&hi2c1, 0x7 << 1, "1", 1, 1000);
```